

Aula 4



JANTAR DOS FILÓSOFOS

O **Problema do Jantar dos Filósofos** (*Dining Philosophers Problem*) é outro problema clássico de computação concorrente formulado por **Edsger Dijkstra** em 1965 (e adaptado posteriormente por Tony Hoare). Ele é amplamente utilizado para ilustrar os desafios de alocação de múltiplos recursos compartilhados, **Deadlock** e **Starvation**.

O Cenário

- **5 filósofos** sentados ao redor de uma mesa redonda.
- Diante de cada filósofo há um prato de espaguete.
- Existem apenas **5 garfos** (ou hashis) dispostos sobre a mesa, um entre cada par de filósofos adjacentes.
- A rotina de cada filósofo consiste em alternar entre duas atividades: **pensar e comer**.
- Para comer, um filósofo **precisa simultaneamente de dois garfos**: o que está à sua esquerda e o que está à sua direita.
- Ao terminar de comer, o filósofo pausa os dois garfos na mesa e volta a pensar.

Os Desafios Concorrentes

Se cada filósofo simplesmente tentar pegar primeiro o garfo à esquerda e depois o da direita:

1. **Deadlock (Impasse Mortal)**: Se todos os 5 filósofos sentirem fome ao mesmo tempo e pegarem o garfo à sua esquerda simultaneamente, nenhum deles conseguirá pegar o garfo à direita. Todos ficarão bloqueados esperando indefinidamente pelo recurso do vizinho.
2. **Starvation (Inanição)**: Mesmo sem um deadlock global, se dois filósofos vizinhos alternarem rapidamente seus momentos de comer, o filósofo que está entre eles pode nunca conseguir os dois garfos ao mesmo tempo e "morrer de fome".
3. **Livelock**: Se os filósofos tentarem contornar o impasse largando o garfo esquerdo ao verem que o direito está ocupado, e todos fizerem isso em sincronia contínua, nenhum come, apesar do sistema continuar ativo e mudando de estado.

Soluções Clássicas

Existem diferentes estratégias para quebrar as condições de deadlock de Coffman (especialmente a *espera circular*):

1. **Assimetria (Quebra da Simetria Circular)**: Filósofos com índices pares pegam primeiro o garfo esquerdo e depois o direito; filósofos ímpares fazem o inverso (pegam primeiro o direito). Isso elimina a possibilidade de dependência circular.
2. **Limite de Capacidade (Controle por Semáforo)**: Permitir que no máximo 4 filósofos sentem-se à mesa ao mesmo tempo (usando um semáforo contador inicializado em 4). Isso garante que ao menos um filósofo sempre conseguirá ambos os garfos.
3. **Aquisição Atômica (Solução de Dijkstra/Tanenbaum)**: Um filósofo só pega os garfos se **ambos** estiverem disponíveis no mesmo instante. Se um estiver ocupado, ele aguarda sem segurar nenhum deles.

BARBEIRO ADORMECIDO

Este problema consiste em manter o barbeiro ocupado enquanto há clientes no estabelecimento, e fazê-lo descansar quando o estabelecimento se encontrar vazio. No estabelecimento há uma cadeira de barbeiro e n cadeiras de espera. Quando não tem clientes, o barbeiro irá sentar na cadeira de barbeiro e ir dormir. Quando um cliente chega, o mesmo deve acordar o barbeiro. No caso da chegada de mais clientes, eles irão esperar caso haja cadeiras de espera vazias, caso contrário irão sair do estabelecimento.

O Desafio de Concorrência

O problema ilustra cenários reais de gerenciamento de recursos limitados (ex.: servidores lidando com filas de requisições de rede ou *thread pools*). Os principais riscos a serem evitados são:

- **Condições de Corrida (*Race Conditions*):** Por exemplo, um cliente verificar que há cadeiras vagas e, antes de conseguir sentar, outro cliente tomar a vaga, gerando inconsistência no contador de assentos.
- **Deadlock (Impasse):** O barbeiro esperar que um cliente o acorde enquanto o cliente pensa que o barbeiro já está acordado esperando por ele.
- **Inanição (*Starvation*):** Garantir que nenhum cliente espere indefinidamente caso a política da fila seja respeitada (usualmente FIFO).

A Solução Clássica (com Semáforos e Mutex)

Para coordenar as *threads* do barbeiro e dos clientes de forma segura, utilizam-se normalmente:

- **mutex (semáforo binário inicializado em 1):** Garante acesso exclusivo à variável compartilhada (o contador de cadeiras ocupadas).
- **clientes (semáforo contador inicializado em 0):** Sinaliza ao barbeiro quantos clientes estão aguardando. Se for zero, o barbeiro bloqueia (dorme).
- **barbeiro (semáforo binário inicializado em 0):** Sinaliza ao cliente que a cadeira de corte está pronta para recebê-lo.

No contexto dos problemas clássicos de concorrência (como o *Jantar dos Filósofos* e o *Barbeiro Adormecido*), um **monitor** é um **mecanismo de sincronização de alto nível** introduzido por C. A. R. Hoare e Per Brinch Hansen.

- **Conceito:** É um módulo ou objeto que encapsula variáveis compartilhadas e as funções (métodos) que operam sobre elas.
- **Exclusão mútua implícita:** Apenas uma *thread* pode executar ativamente código dentro do monitor em um determinado instante. Se outra *thread* tentar chamar um método do monitor enquanto ele estiver ocupado, ela é automaticamente bloqueada pelo compilador/runtime, sem que o programador precise manipular primitivas de baixo nível (como `wait` e `signal` em semáforos manuais).
- **Variáveis de condição (*Condition Variables*):** Para permitir sincronização e coordenação de espera, monitores utilizam operações como:
 - `wait()`: Libera o acesso ao monitor e coloca a *thread* em espera até ser notificada.
 - `signal()` ou `notify()`: Acorda uma das *threads* que estavam esperando na variável de condição.
- **Exemplo prático:** O modificador `synchronized` do Java implementa a semântica básica de um monitor em torno de objetos.